Ejercicios de Pipeline resueltos

> **Hipótesis comunes**
> Salvo que se indique lo contrario, todos los ejercicios suponen el pipeline clásico de 5 etapas descrito en este documento: memorias de instrucciones y datos separadas; banco de registros que escribe en la primera mitad del ciclo y lee en la segunda; detenciones en ID; saltos resueltos en EX; predicción estática «no tomado».

Ejercicio 1. Procesador monociclo frente a segmentado

Enunciado. Las etapas de un procesador RISC-V tienen estas latencias: IF = 200 ps, ID = 100 ps, EX = 200 ps, MEM = 200 ps y WB = 100 ps.

a) Calcula el tiempo de ciclo de una implementación monociclo y de una segmentada.

b) Calcula el tiempo necesario para ejecutar 1000 instrucciones sin riesgos en cada caso y la aceleración obtenida.

c) Repite el apartado b) si cada registro de segmentación añade 20 ps.

Solución

a) En la versión monociclo el ciclo debe dar tiempo a la instrucción más lenta, `lw`, que usa las cinco etapas: 200 + 100 + 200 + 200 + 100 = **800 ps**. En la segmentada, el ciclo lo marca la etapa más lenta: **200 ps**.

b) Monociclo: 1000 × 800 ps = 800 000 ps = **800 ns**. Segmentado: (5 + 1000 − 1) × 200 ps = 1004 × 200 ps = 200 800 ps ≈ **200,8 ns**. Aceleración: S = 800 000 / 200 800 ≈ **3,98**.

c) Ciclo = 200 + 20 = 220 ps → 1004 × 220 ps = 220 880 ps ≈ **220,9 ns**. Aceleración: S = 800 000 / 220 880 ≈ **3,62**.

Conclusión. No se alcanza la aceleración ideal de 5 porque las etapas no están equilibradas (ID y WB sólo necesitan 100 ps de los 200 del ciclo, por lo que el máximo posible es 800 / 200 = 4) y porque los registros de segmentación añaden retardo.

Ejercicio 2. Dependencias y diagrama con y sin anticipación

Enunciado. Dado el siguiente fragmento:

```
I1:  add  x5, x6, x7
I2:  sub  x8, x5, x9
I3:  lw   x10, 0(x8)
I4:  or   x11, x10, x5
I5:  sw   x11, 4(x8)
```

a) Identifica las dependencias RAW.

b) Dibuja el diagrama multiciclo sin anticipación y calcula el número de ciclos.

c) Repite el apartado b) con anticipación.

Solución

a) Dependencias RAW:

Productora → consumidora	Registro	Distancia	Comentario
I1 → I2	x5	1	ALU → ALU
I2 → I3	x8	1	x8 es la base de la dirección de `lw`
I3 → I4	x10	1	**Carga-uso**
I4 → I5	x11	1	x11 es el dato que guarda `sw`
I1 → I4	x5	3	No provoca riesgo
I2 → I5	x8	3	No provoca riesgo

b) Sin anticipación, cada dependencia a distancia 1 cuesta 2 ciclos: 4 × 2 = 8 detenciones. Total = 5 + (5 − 1) + 8 = **17 ciclos**.

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
`add x5, x6, x7`	IF	ID	EX	MEM	WB												
`sub x8, x5, x9`		IF	ID	*ID*	*ID*	EX	MEM	WB									
`lw x10, 0(x8)`			IF	*IF*	*IF*	ID	*ID*	*ID*	EX	MEM	WB						
`or x11, x10, x5`						IF	*IF*	*IF*	ID	*ID*	*ID*	EX	MEM	WB			
`sw x11, 4(x8)`									IF	*IF*	*IF*	ID	*ID*	*ID*	EX	MEM	WB

detención (la instrucción repite etapa)

Diagrama E2-a. Sin anticipación: 8 detenciones, 17 ciclos.

c) Con anticipación sólo queda la carga-uso I3 → I4, que cuesta 1 ciclo. El resto se resuelve anticipando desde EX/MEM: I1 → I2, I2 → I3 e I4 → I5 (el valor de x11 viaja con `sw` hasta MEM). Total = 9 + 1 = **10 ciclos**.

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10
`add x5, x6, x7`	IF	ID	EX	MEM	WB					
`sub x8, x5, x9`		IF	ID	EX	MEM	WB				
`lw x10, 0(x8)`			IF	ID	EX	MEM	WB			
`or x11, x10, x5`				IF	ID	*ID*	EX	MEM	WB	
`sw x11, 4(x8)`					IF	*IF*	ID	EX	MEM	WB

detención (la instrucción repite etapa)

Diagrama E2-b. Con anticipación: 1 detención, 10 ciclos. Aceleración debida a la anticipación: 17 / 10 = 1,7.

Ejercicio 3. Clasificación de dependencias: RAW, WAR y WAW

Enunciado. Para el siguiente código:

```
I1:  lw   x5, 0(x10)
I2:  add  x6, x5, x7
I3:  sub  x5, x8, x9
I4:  and  x10, x6, x5
I5:  or   x7, x10, x5
```

a) Clasifica todas las dependencias entre instrucciones.
b) Indica cuáles provocan riesgos en el pipeline de 5 etapas.
c) Calcula el número de ciclos sin anticipación y con ella.

Solución

a)

Tipo	Instrucciones	Registro	Explicación
RAW	I1 → I2	x5	I2 lee el x5 que carga I1
RAW	I2 → I4	x6	I4 lee el x6 que calcula I2
RAW	I3 → I4	x5	I4 lee el x5 de I3 (el más reciente, no el de I1)
RAW	I3 → I5	x5	I5 lee el x5 de I3
RAW	I4 → I5	x10	I5 lee el x10 que calcula I4
WAR	I1 → I4	x10	I1 lee x10 (base) e I4 lo sobrescribe
WAR	I2 → I3	x5	I2 lee x5 e I3 lo sobrescribe
WAR	I2 → I5	x7	I2 lee x7 e I5 lo sobrescribe
WAW	I1 → I3	x5	Ambas escriben x5; debe quedar el valor de I3

b) Sólo las RAW. Las WAR y WAW no causan problemas porque las instrucciones leen en ID y escriben en WB siempre en orden de programa.

c) Sin anticipación: I2 espera 2 ciclos (x5 de `lw`), I4 espera 2 (x5 de I3, a distancia 1) e I5 espera 2 (x10 de I4): 6 detenciones → 9 + 6 = **15 ciclos**.

Con anticipación sólo queda la carga-uso I1 → I2 (1 ciclo) → **10 ciclos**. En el ciclo 7, I4 recibe x6 desde MEM/WB (I2) y x5 desde EX/MEM (I3), una por cada entrada de la ALU; en el ciclo 8, I5 recibe x10 desde EX/MEM (I4) y x5 desde MEM/WB (I3).

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10
`lw x5, 0(x10)`	IF	ID	EX	MEM	WB					
`add x6, x5, x7`		IF	ID	*ID*	EX	MEM	WB			
`sub x5, x8, x9`			IF	*IF*	ID	EX	MEM	WB		
`and x10, x6, x5`					IF	ID	EX	MEM	WB	
`or x7, x10, x5`						IF	ID	EX	MEM	WB

detención (la instrucción repite etapa)

Diagrama E3. Con anticipación: una sola detención (carga-uso), 10 ciclos.

Ejercicio 4. Reordenación de código para evitar riesgos de carga-uso

Enunciado. El compilador traduce `a = b + e; c = b + f;` (variables en memoria, dirección base en x31) así:

```
lw   x1, 0(x31)      # b
lw   x2, 8(x31)      # e
add  x3, x1, x2      # a = b + e
sw   x3, 24(x31)     # guarda a
lw   x4, 16(x31)     # f
add  x5, x1, x4      # c = b + f
sw   x5, 32(x31)     # guarda c
```

a) Con anticipación, ¿qué riesgos hay y cuántos ciclos tarda?

b) Reordena el código para eliminar todas las detenciones y calcula los ciclos.

Solución

a) Hay dos riesgos de carga-uso: `lw x2` → `add x3` y `lw x4` → `add x5`. Cada uno cuesta 1 ciclo: 5 + 6 + 2 = **13 ciclos**.

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw x1, 0(x31)`	IF	ID	EX	MEM	WB								
`lw x2, 8(x31)`		IF	ID	EX	MEM	WB							
`add x3, x1, x2`			IF	ID	*ID*	EX	MEM	WB					
`sw x3, 24(x31)`				IF	*IF*	ID	EX	MEM	WB				
`lw x4, 16(x31)`						IF	ID	EX	MEM	WB			
`add x5, x1, x4`							IF	ID	*ID*	EX	MEM	WB	
`sw x5, 32(x31)`								IF	*IF*	ID	EX	MEM	WB

detención (la instrucción repite etapa)

Diagrama E4-a. Código original: 2 detenciones, 13 ciclos.

b) Basta con subir `lw x4` detrás de `lw x2`. El cambio es correcto porque `lw x4` no depende de `add x3` ni de `sw x3`, y lee una dirección de memoria (16(x31)) distinta de la que escribe `sw` (24(x31)):

```
lw   x1, 0(x31)
lw   x2, 8(x31)
lw   x4, 16(x31)      # subida dos posiciones
add  x3, x1, x2       # x2 ya está a distancia 2: anticipación MEM/WB
sw   x3, 24(x31)
add  x5, x1, x4       # x4 a distancia 3: sin riesgo
sw   x5, 32(x31)
```

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw x1, 0(x31)`	IF	ID	EX	MEM	WB								
`lw x2, 8(x31)`		IF	ID	EX	MEM	WB							
`lw x4, 16(x31)`			IF	ID	EX	MEM	WB						
`add x3, x1, x2`				IF	ID	EX	MEM	WB					
`sw x3, 24(x31)`					IF	ID	EX	MEM	WB				
`add x5, x1, x4`						IF	ID	EX	MEM	WB			
`sw x5, 32(x31)`							IF	ID	EX	MEM	WB		

Diagrama E4-b. Código reordenado: sin detenciones, 11 ciclos. Aceleración: 13 / 11 ≈ 1,18.

Ejercicio 5. Salto tomado: resolución en EX frente a resolución en ID

Enunciado. Con predicción estática «no tomado»:

```
I1:        addi x10, x0, 3
I2:        bne  x10, x0, SIG    # x10 = 3 ≠ 0 → el salto SE TOMA
I3:        add  x11, x10, x10   # camino incorrecto
I4:        sub  x12, x10, x10   # camino incorrecto
           ...
I5:  SIG:  or   x13, x10, x10
I6:        and  x14, x13, x10
```

a) Dibuja el diagrama si el salto se resuelve en EX y calcula los ciclos hasta que termina I6.

b) Repite el apartado a) si el salto se resuelve en ID. ¿Mejora el resultado?

c) ¿Cuántos ciclos tardaría si un BTB predijera correctamente el salto como tomado?

Solución

a) `bne` recibe x10 por anticipación EX/MEM → EX en el ciclo 4 y se resuelve al final de ese ciclo. I3 (en ID) e I4 (en IF) se anulan e I5 se busca en el ciclo 5. Se pierden 2 ciclos: total = **10 ciclos**.

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10
`addi x10, x0, 3`	IF	ID	EX	MEM	WB					
`bne  x10, x0, SIG`		IF	ID	EX	MEM	WB				
`add  x11, x10, x10`			~~IF~~	~~ID~~						
`sub  x12, x10, x10`				~~IF~~						
`SIG: or x13, x10, x10`					IF	ID	EX	MEM	WB	
`and  x14, x13, x10`						IF	ID	EX	MEM	WB

instrucción anulada (flush)

Diagrama E5-a. Salto resuelto en EX: 2 instrucciones anuladas, 10 ciclos.

b) Para resolver en ID, `bne` necesita x10 en el ciclo 3, pero `addi` lo calcula al final de ese mismo ciclo. Hay que detener `bne` 1 ciclo y anticipar x10 hacia ID en el ciclo 4. El salto se resuelve al final del ciclo 4 y sólo se anula I3, que estaba esperando en IF. Total = **10 ciclos**: los mismos que en a).

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9	10
`addi x10, x0, 3`	IF	ID	EX	MEM	WB					
`bne  x10, x0, SIG`		IF	ID	*ID*	EX	MEM	WB			
`add  x11, x10, x10`			~~IF~~	~~IF~~						
`SIG: or x13, x10, x10`					IF	ID	EX	MEM	WB	
`and  x14, x13, x10`						IF	ID	EX	MEM	WB

detención (la instrucción repite etapa) instrucción anulada (flush)

Diagrama E5-b. Salto resuelto en ID: 1 detención por la dependencia + 1 instrucción anulada = 2 ciclos perdidos.

Resolver en ID sólo ahorra ciclos cuando el salto no depende de la instrucción inmediatamente anterior. Aquí la dependencia `addi` → `bne` se come la ventaja. Si el compilador separase ambas instrucciones con otra independiente, la penalización bajaría a 1 ciclo.

c) Si el BTB predice «tomado» y proporciona el destino en IF, SIG se busca en el ciclo 3, justo detrás de `bne`, y no se pierde ningún ciclo: I1, I2, I5 e I6 terminan en 5 + 3 = **8 ciclos**.

Ejercicio 6. Cálculo del CPI con riesgos de datos y de control

Enunciado. Un programa tiene esta mezcla de instrucciones: cargas 25 % (el 40 % de ellas van seguidas de una instrucción que usa el dato cargado), saltos condicionales 15 % (el 60 % se toman) y el resto aritmético-lógicas. Hay anticipación completa y los saltos se resuelven en EX (penalización de 2 ciclos). Calcula el CPI:

a) Si el pipeline se detiene en todos los saltos hasta resolverlos.
b) Con predicción estática «no tomado».
c) Con un predictor dinámico que acierta el 90 % (con BTB, de modo que los aciertos no cuestan ciclos).
d) ¿Cuánto más rápido es c) que a)? ¿Cuánto tarda c) en ejecutar 10^9 instrucciones a 2 GHz?

Solución

La aportación de las cargas es la misma en todos los casos: 0,25 × 0,40 × 1 = **0,10** ciclos por instrucción.

Caso	Aportación de los saltos	CPI
a) Detener siempre	0,15 × 2 = 0,30	**1 + 0,10 + 0,30 = 1,40**
b) Predicción «no tomado»	0,15 × 0,60 × 2 = 0,18	**1 + 0,10 + 0,18 = 1,28**
c) Predictor dinámico (90 %)	0,15 × 0,10 × 2 = 0,03	**1 + 0,10 + 0,03 = 1,13**

d) Aceleración de c) frente a a): 1,40 / 1,13 ≈ **1,24** (un 24 % más rápido). Con un ciclo de 0,5 ns: T = 10^9 × 1,13 × 0,5 ns ≈ **0,565 s** (frente a 0,70 s en el caso a).

Ejercicio 7. Predictores de 1 bit y de 2 bits en un bucle

Enunciado. En el siguiente código, el bucle interno se ejecuta 10 veces cada vez que se entra en él, y se entra en él muchas veces:

```
externo:  addi x5, x0, 10        # contador del bucle interno
interno:  ...                    # cuerpo del bucle
          addi x5, x5, -1
          bne  x5, x0, interno   # salto hacia atrás
          addi x6, x6, -1
          bne  x6, x0, externo
```

Para el salto `bne x5, x0, interno`, calcula en régimen estacionario el número de fallos y la tasa de aciertos por cada ejecución del bucle interno con: predicción «no tomado», predicción estática BTFN, predictor de 1 bit y predictor de 2 bits. Supón que, cuando se predice «tomado», el BTB da el destino en IF, y que la penalización por fallo es de 2 ciclos.

Solución

En cada ejecución del bucle interno el salto se toma 9 veces (iteraciones 1–9) y no se toma 1 vez (iteración 10).

Predictor de 1 bit. Al salir del bucle el bit queda en «no tomado». En la siguiente entrada:

- Iteración 1: predice NT, se toma → **fallo**; el bit pasa a T.
- Iteraciones 2–9: predice T, se toma → aciertos.
- Iteración 10: predice T, no se toma → **fallo**; el bit pasa a NT.

Predictor de 2 bits. Al salir, el contador pasa de 11 a 10, que sigue prediciendo «tomado»:

- Iteración 1: estado 10 predice T, se toma → acierto; pasa a 11.
- Iteraciones 2–9: estado 11 predice T → aciertos.
- Iteración 10: estado 11 predice T, no se toma → **fallo**; pasa a 10.

Predictor	Fallos por ejecución del bucle	Tasa de aciertos	Ciclos perdidos
Estático «no tomado»	9	10 %	18
Estático BTFN (hacia atrás: tomado)	1	90 %	2
Dinámico de 1 bit	2	80 %	4
Dinámico de 2 bits	**1**	**90 %**	**2**

Para este salto concreto BTFN iguala al predictor de 2 bits, pero el de 2 bits también se adapta a saltos hacia delante con comportamiento sesgado, cosa que BTFN no hace.

Ejercicio 8. Optimización completa de un bucle: suma de un vector

Enunciado. El siguiente código suma los n = 100 elementos de un vector:

```
# x10 = dirección de v[0], x11 = n = 100; el resultado queda en x12
          addi x12, x0, 0        # suma = 0
bucle:    lw   x5, 0(x10)        # x5 = v[i]
          add  x12, x12, x5      # suma += v[i]
          addi x10, x10, 4       # siguiente elemento
          addi x11, x11, -1      # n--
          bne  x11, x0, bucle
```

a) Identifica los riesgos de una iteración.
b) Calcula los ciclos totales y el CPI con predicción «no tomado».
c) Reordena el bucle para eliminar las detenciones por datos y recalcula.
d) Recalcula c) con un predictor dinámico con BTB que sólo falla en la primera y en la última ejecución del salto.

Solución

a) `lw x5` → `add x12` es un riesgo de carga-uso (1 detención por iteración). `addi x11` → `bne` es una dependencia ALU → salto a distancia 1, que la anticipación EX/MEM → EX resuelve sin

detención (con resolución en ID costaría 1 ciclo más). Además, `bne` se toma 99 de las 100 veces, y con predicción «no tomado» cada salto tomado cuesta 2 ciclos.

b) Se ejecutan 1 + 5 × 100 = 501 instrucciones. En régimen estacionario cada iteración cuesta 5 + 1 + 2 = 8 ciclos.

Ciclos = 501 + 4 (llenado) + 100 × 1 (carga-uso) + 99 × 2 (saltos) = 803 CPI = 803 / 501 ≈ 1,60

c) Se adelantan las dos `addi` para separar la carga de su uso. Es correcto porque `lw` lee x10 antes de que `addi` lo modifique (dependencia WAR que se respeta en orden):

```
bucle:     lw   x5, 0(x10)
           addi x10, x10, 4
           addi x11, x11, -1
           add  x12, x12, x5      # lw está ahora a distancia 3
           bne  x11, x0, bucle    # addi x11 a distancia 2
```

Ciclos = 501 + 4 + 99 × 2 = 703 CPI = 703 / 501 ≈ 1,40

d) Con el predictor sólo se pierden 2 ciclos en la primera ejecución del salto (el BTB aún no lo conoce) y 2 en la última (sale del bucle):

Ciclos = 501 + 4 + 2 × 2 = 509 CPI = 509 / 501 ≈ 1,02

Aceleración total frente al caso b): 803 / 509 ≈ **1,58**. La reordenación elimina las pérdidas por datos y la predicción, casi todas las de control.